

Vite & Gourmand Documentation de gestion de projet

ECF Studi — TP Développeur Web et Web Mobile

1. Contexte et méthode

Le projet Vite & Gourmand a été mené en autonomie complète, de l'analyse des besoins jusqu'au déploiement en production. En tant que développeur unique, j'ai adopté une méthode de gestion de projet légère mais rigoureuse, inspirée des principes agiles : un découpage du travail en tâches unitaires, un suivi visuel de l'avancement, et une priorisation continue.

L'objectif était de conserver à tout moment une vision claire de ce qui restait à faire, d'éviter les oublis fonctionnels par rapport au cahier des charges, et de pouvoir ajuster les priorités au fil de l'avancement.

2. Outil utilisé : Notion

J'ai choisi Notion comme outil de gestion de projet, sous la forme d'une base de données de tâches (« Suivi des tâches — Vite & Gourmand »). Ce choix se justifie par sa souplesse : Notion permet de combiner une liste de tâches, des vues filtrées (par statut, par domaine) et une vue calendrier, tout en restant gratuit et accessible depuis n'importe quel appareil.

Structure de la base

Propriété	Rôle
Tâche	Intitulé clair et atomique de la tâche à réaliser
Status	Not started / In progress / Terminé — avancement visuel
Priority	High / Medium / Low — priorisation du travail
Domaine	Regroupement par fonctionnalité (voir section 3)
Échéance	Date cible optionnelle

Trois vues facilitent le pilotage : All tasks (liste complète), By status (regroupement Kanban par état d'avancement) et Calendrier (échéances).

3. Découpage par domaines fonctionnels

Le travail a été organisé en domaines correspondant aux grandes fonctionnalités de l'application. Ce découpage présente un avantage clé : il correspond directement à la stratégie de branches Git du projet (une branche feature/* par domaine), assurant une cohérence totale entre la gestion de projet et le versionnage du code.

Domaine Notion	Branche Git correspondante
Setup & environnement	(préparation initiale, README, structure)
Authentification & rôles	feature/authentification
Gestion des menus	feature/gestion-menus
Commandes	feature/commandes
Espace utilisateur	feature/espace-utilisateur
Espace employé	feature/espace-employe
Espace administrateur	feature/espace-admin
Front-end	(pages publiques : accueil, contact, mentions...)
Base de données	(MCD, fichiers SQL, connexions MySQL/MongoDB)
Déploiement	(Vercel, Render, Clever Cloud, MongoDB Atlas)
Maquettes & charte graphique	(wireframes, mockups, charte)
Documentation	(manuel, doc technique, diagrammes)

4. Priorisation

Les tâches ont été priorisées en trois niveaux pour concentrer l'effort sur l'essentiel :

- High : fonctionnalités cœur du cahier des charges (authentification, commandes, gestion des menus, espaces par rôle, déploiement, livrables documentaires obligatoires).
- Medium : fonctionnalités importantes mais secondaires (avis clients, calcul des frais de livraison, vérification du stock, mockups haute fidélité).
- Low : pages de contenu statique (mentions légales, conditions générales de vente).

Cette hiérarchisation a permis de garantir qu'une version fonctionnelle et conforme était disponible avant de traiter les éléments de moindre priorité.

5. Déroulement et suivi

Le projet a suivi un enchaînement logique : mise en place de l'environnement et du dépôt Git, conception (MCD, maquettes, charte), puis développement fonctionnalité par fonctionnalité, chacune développée sur sa branche dédiée avant intégration. Le déploiement a été réalisé progressivement, puis l'application a été finalisée par une phase de sécurisation, d'accessibilité (RGAA) et de documentation.

Le statut de chaque tâche était mis à jour au fil de l'eau dans Notion, offrant à tout moment une photographie de l'avancement global et des points restants.

6. Difficultés rencontrées et solutions

Plusieurs difficultés techniques ont été rencontrées et résolues au cours du projet :

- Cookies cross-origin (Chrome) : le blocage des cookies tiers entre le front (Vercel) et le back (Render), sur des domaines différents, a nécessité une analyse approfondie. La solution mise en place est un proxy Vercel : les appels `/api/*` du front sont réécrits côté serveur vers l'API Render, plaçant les deux applications sur un même domaine du point de vue du navigateur. Cela a permis de passer les cookies de `SameSite=None` à `SameSite=Lax`, valeur plus restrictive et plus sûre contre le CSRF.
- Documentation Swagger et CSP : la mise en place d'une Content Security Policy stricte bloquait l'affichage de Swagger UI ; la solution a consisté à exclure la seule route `/api/doc` de la CSP via un request matcher, préservant la sécurité du reste de l'application. La mise en place du proxy a ensuite révélé, via des 404 sur les assets de Swagger UI, que ceux-ci sont servis par l'AssetMapper de Symfony sous `/assets/*` (et non `/bundles/*` comme dans les versions antérieures du bundle) ; l'ajout de cette règle de réécriture au proxy a rétabli l'affichage complet.
- Déploiement Docker sur Render : l'ajustement du port Apache (10000) attendu par le scan de ports de Render, et la gestion des assets, ont demandé plusieurs itérations.
- Refactorisation : l'extraction de la logique métier des contrôleurs vers des services dédiés (CommandeService) pour corriger l'anti-pattern du « Fat Controller ».

Ces difficultés, tracées et résolues méthodiquement, illustrent une démarche de développement itérative et une capacité à diagnostiquer puis corriger des problèmes techniques réels.

Lien du board Notion : <https://www.notion.so/7bbf6c33689f48798cac103ce1ece2a3?v=7f5ec6945ca4454ca6b9e22069144fa5>